# Wizard plugin framework spike

## What & Why

This is a **spike + proof-of-concept** task. The goal is to design and build a new "wizard as plugin" framework for the codebase, prove it out by porting one existing wizard end-to-end, and leave every other wizard untouched on the existing system. No mass migration happens in this task — the migration of the remaining ~17 wizards will be planned as follow-up work *after* the team has seen the framework in action and agreed it's the right shape.

### The problem we're trying to solve

Today, adding or modifying a wizard touches an uncomfortable number of unrelated files in unrelated parts of the tree. The "wizard" is not really a single unit — it's scattered across three layers that each speak a different language:

1. **`server/wizards/types/<name>.ts`** — a class that declares step metadata (`id`, `name`, `description`) but contains *no* behavior. It's purely a manifest masquerading as code.
2. **`server/modules/<area>/<name>.ts`** — hand-rolled Express routes (one per step action), each re-implementing the same boilerplate:
   - `requireAuth` / `requirePermission(...)` checks
   - request body validation (often ad-hoc, not always with Zod)
   - loading the wizard by id
   - calling `storage.wizards.update(...)` with the new step + status + data
   - error handling + logging
   - in some cases, `multer` setup for file uploads
3. **`client/src/pages/wizard-view.tsx`** plus per-wizard page files under `client/src/pages/...` — the UI for each step is either inlined into the shared wizard-view dispatcher or lives in a per-wizard page that mostly exists to list previous runs and create new ones.

Concretely, for `BtuBuildingRepImportWizard` the four steps (`upload`, `preview`, `process`, `results`) are described in `server/wizards/types/btu_building_rep_import.ts`, but the actual logic for `process` lives in `server/modules/sitespecific/btu/building-rep-import.ts` behind `POST /api/btu-building-rep-import/process`, and a developer looking at the step list has no way to

know that without grepping the route file. This is representative of every wizard in the system.

The pain this causes:

- **High blast radius for new wizards.** Adding one wizard means: edit the types folder, add a module file, register routes in the routes index, edit `wizard-view.tsx` to render the right step UI, often add a landing page under `client/src/pages/...`, and wire that page into `App.tsx`. That's six+ edits for what should be one feature. This is a strong disincentive against refactoring wizards or splitting overgrown ones.
- **Inconsistent permission / validation enforcement.** Each per-wizard module re-declares its own `requirePermission(...)` and its own input validation. An auditor asking "can a non-admin user hit any wizard step they shouldn't?" has to read every per-wizard module file and check every handler individually. There is no single place that enforces this invariant. This directly conflicts with the broader effort (see recent storage / audit-log consolidation tasks) to shrink the surface area that has to be audited.
- **Inconsistent wizard-state persistence.** Some routes call `storage.wizards.update(...)` with `currentStep` and `status` correctly, some forget one or the other, some forget to merge `wizard.data` and accidentally clobber prior step output. Centralizing this is a correctness win, not just a tidiness win.
- **The "type" file lies about what it is.** Calling `server/wizards/types/<name>.ts` a wizard *type* implies it owns the wizard's behavior. It does not. It owns four strings per step. New contributors routinely look there first for the logic and bounce off.
- **Frontend has to be edited for every new wizard.** `wizard-view.tsx` (or its per-step children) must be taught about every new wizard type. There is no registration mechanism — it's an `if`/`switch` over wizard types. This is the frontend mirror of the backend routes problem.

## What "good" looks like

A wizard should be **one logical plugin**, congruent with the four existing server plugin architectures (`server/plugins/trust/eligibility`, `server/plugins/ledger/charge`, `server/plugins/dispatch/eligibility`, `server/plugins/dashboard`). That means:

- The plugin lives **entirely on the server**, in a single folder under `server/plugins/wizards/plugins/<name>/`, matching the existing plugin convention (e.g. `server/plugins/dashboard/plugins/...`, `server/plugins/trust/eligibility/plugins/...`).
- Adding a wizard means creating one server plugin folder and one client step-components folder. No edits to routes, no edits to `wizard-view.tsx`, no edits to `App.tsx`,

no new file under `shared/`.

- Common step shapes (file upload, review/confirm, results display) are available as **prebuilt generic steps** under `server/plugins/wizards/generic/` and `client/src/plugins/wizards/generic/`, so wizards that only need standard step UIs don't write any React at all — they just compose generic steps into their step map.
- The server's manifest response **declares which React component renders each step**, by string identifier (`"generic:Review"`, `"btu_building_rep_import:Process"`). A custom server handler can declare a generic client component, which is the common case: most wizards have bespoke validation/mapping/processing logic on the server but a generic upload / review / results UI on the client.
- The client component registry is **auto-discovered** via `import.meta.glob` from `client/src/plugins/wizards/`. There is no per-wizard `index.ts` to maintain. A wizard whose UI is entirely generic ships **zero** client-side files.
- The set of HTTP routes the security team has to audit for wizards is **fixed**. Adding wizards does not grow it.
- Permission, validation, file-upload handling, error handling, logging, and wizard-state persistence happen in **one place** (the dispatcher), driven by declarative metadata that each step exports.
- The "what does this wizard do" question is answered by reading **one server folder**, not by spelunking through three trees.
- The existing project rule — *all DB access goes through* `server/storage/` — is preserved verbatim. Wizard plugin handlers may only call `storage.*`; they may not import `db`, `getClient`, schema tables, or `drizzle-orm` query operators.

### Why no `shared/` manifest

An earlier draft of this spike proposed a `shared/wizards/<name>/manifest.ts` that both server and client would import directly. We are explicitly **not** doing that, for three reasons:

1. **Directory hygiene.** It would add a new top-level convention under `shared/` and a new tree to keep in sync. We have enough trees.
2. **Congruence with existing plugin architectures.** The four existing server-side plugin systems all live entirely under `server/plugins/` with no shared half. A wizard plugin should look the same. Anyone who has written a dispatch eligibility plugin or a dashboard plugin should recognize the layout immediately.
3. **Manifests are sometimes dynamic.** A wizard's step list may depend on `wizard.data` plus storage lookups (e.g. an onboarding wizard that skips the "facility selection" step

when the user is already scoped to one facility, or a report wizard whose available export formats depend on which components are enabled). A static `shared/` file literally cannot express that — it would have to be a function — and once it's a function that runs storage queries, it belongs on the server.

The framework therefore keeps the plugin on the server and lets the client **fetch** whatever manifest information it needs over HTTP. The same endpoint that returns a wizard returns its computed manifest. The client never imports plugin code; it consumes a JSON response.

### Why this is a spike, not a full migration

There are 18 wizards in `server/wizards/types/` today, several with non-trivial flows (BTU dues allocation, GBHET legal, employer onboarding, several report wizards). A "stop the world and rewrite everything" task is high-risk: if the framework design turns out to be wrong in some subtle way, we'd be unwinding a huge amount of work.

The spike approach is:

1. Build the framework alongside the existing system. Old wizards keep working exactly as they do today.
2. Port **one** wizard (`btu_building_rep_import`, chosen because it's small, has all four common step kinds — upload, review, process, results — and is already well-understood) end-to-end onto the new framework.
3. Stop. Team uses the result, reviews the code, and decides whether to green-light migrating the remaining 17 wizards.

If the design is wrong, we've spent one task instead of ten, and the existing wizards are untouched.

## Done looks like

When this task is complete:

- A new generic wizard dispatcher exists on the server, exposing a small, fixed set of routes that handle *any* wizard plugin. Adding new wizards does not add new routes.
- A new generic `WizardView` exists on the client that renders the step strip and the active step UI based on data fetched from the server. Adding new wizards does not require editing `WizardView`.
- A server-side wizard plugin registry exists at `server/plugins/wizards/`, congruent with the existing `server/plugins/trust/eligibility`, `server/plugins/ledger/charge`, `server/plugins/dispatch/eligibility`, and `server/plugins/dashboard` plugin systems.

Individual wizard plugins live under `server/plugins/wizards/plugins/<name>/` (the `plugins/plugins/` nesting is the existing project convention — see the four systems above).

- A library of **generic step handlers** exists at `server/plugins/wizards/generic/<stepId>.ts`, providing prebuilt implementations of common step shapes (upload, review, results display) that any wizard plugin can compose into its step map.
- A client-side component registry exists at `client/src/plugins/wizards/registry.ts`, **auto-discovered** via `import.meta.glob` from sibling folders. There is no per-wizard `index.ts`. Components are resolved by string identifier (`"<namespace>:<ComponentName>"`), and the namespace + filename are the identifier. A wizard whose UI is fully generic ships no client-side files at all. A library of default components for the generic step ids lives at `client/src/plugins/wizards/generic/` (`Upload.tsx`, `Review.tsx`, `Results.tsx`, registered as `"generic:Upload"`, etc.). No client-side declarations about permissions, kinds, step lists, or component-to-step bindings — all of that comes from the server's manifest response.
- `BtuBuildingRepImportWizard` has been fully ported to the new framework:
  - Its old per-wizard routes in `server/modules/sitespecific/btu/building-rep-import.ts` have been removed (the file may be deleted if nothing else lives there).
  - Its frontend page now goes through the generic `WizardView`. The landing-page file may remain as the index/listing page; only the step-rendering responsibility moves.
  - The wizard still works end-to-end from the user's perspective: upload CSV → preview matches → process assignments → see results.
  - The old `server/wizards/types/btu_building_rep_import.ts` is removed so there is one source of truth for that wizard's step list.
- The other 17 wizards in `server/wizards/types/` are **untouched** and continue to work exactly as they do today. The two systems coexist.
- There is a short developer note (added to `replit.md` under "Where things live" or a new "Wizards" section) describing the new layout and how to add a wizard under it, with a pointer to `btu_building_rep_import` as the canonical example.

## Out of scope

- Migrating any wizard other than `btu_building_rep_import`. The other 17 stay on the

existing system. Their migration will be planned as separate follow-up tasks **after** the team reviews the spike result.

- Removing the existing wizard infrastructure (the old `server/wizards/types/` base classes, the old per-wizard modules, the old branches in `wizard-view.tsx`). All of that has to stay in place to keep the other wizards running. It will be cleaned up at the *end* of the eventual migration, not here.
- Replacing the wizard create endpoint or any cross-wizard infrastructure that already works (statuses, `wizards` table schema, the `/wizards/:id` route shell). Those stay as-is. The new framework adds endpoints; it does not remove existing ones.
- Introducing JSON-Schema / RJSF forms. The framework should leave the door open for a future schema-driven `form` step kind, but no actual schema-rendered forms ship in this task. Every step in the ported wizard uses a hand-written React component.
- Any change to the existing storage-layer rule. Plugin handlers still call `storage.*` and only `storage.*`. The dispatcher does not introduce a new way to access the database.
- Any `shared/wizards/` directory. Manifest information is computed on the server and shipped to the client over HTTP.

## Design notes

These are constraints the implementer should respect; they are the outcome of the discussion that produced this spike. They are intentionally specific because the framework's *shape* is the thing we want the team to react to.

### Folder layout

A wizard plugin lives entirely on the server, mirroring the existing `server/plugins/<area>/plugins/<name>/` convention. The client carries only the React step components, mirroring the existing `client/src/plugins/<area>/` convention. The client does not import any plugin metadata:

```
server/plugins/wizards/
  registry.ts                       # wizard plugin registry (parallels
server/plugins/dashboard/registry.ts)
  types.ts                          # WizardManifest, WizardStepHandler, etc.
  dispatcher.ts                     # the fixed-route HTTP dispatcher
  generic/                          # prebuilt generic step handlers, importable by any
plugin
    upload.ts
```

```
    review.ts
    results.ts
  plugins/
    <name>/
      index.ts                        # registers the plugin; exports { type, manifest,
steps }
      steps/
        <stepId>.ts                   # custom step handler (only when a generic step
won't do)

client/src/plugins/wizards/
  registry.ts                         # auto-discovered component registry (uses
import.meta.glob)
  WizardView.tsx                      # generic wizard renderer
  generic/                            # default React components, registered as
"generic:<Name>"
    Upload.tsx                        # -> "generic:Upload"
    Review.tsx                        # -> "generic:Review"
    Results.tsx                       # -> "generic:Results"
  <name>/                             # only created if this wizard needs custom React
components
    <ComponentName>.tsx               # -> "<name>:<ComponentName>"  (no index.ts
required)
```

There is **no** `shared/wizards/` directory. There is **no** shared manifest file. The client learns about a wizard's steps by calling the server.

The `plugins/plugins/` nesting on the server side is intentional and matches the four existing plugin systems — it gives each plugin area room for its own `registry.ts`, `types.ts`, and (here) `generic/` and `dispatcher.ts` siblings next to the actual plugin folders.

### Server plugin shape

Each plugin exports:

- A `type` string matching the `wizards.type` column (e.g. `'btu_building_rep_import'`).
- A `manifest` function (sync or async) of the form:

  ```
  async function manifest({ wizard, storage, user }): Promise<WizardManifest> { ...
  }
  ```

...returning the displayable wizard metadata plus the **current** step list for this wizard (which may depend on `wizard.data` or storage lookups). The manifest contains:

  - `displayName`, `description`
  - `requiredComponent` (top-level component gate for the wizard)
  - `steps: WizardStepManifest[]`, where each entry includes:
    - `id`, `name`, `description`
    - `kind` (`'upload' | 'review' | 'results' | 'custom' | 'form'`) — used by the dispatcher to decide whether to mount multer, **not** by the client to pick a component
    - `requiredPermission`
    - `state` (`'pending' | 'available' | 'current' | 'complete'`) computed from `wizard.currentStep` and any plugin-specific logic
    - `component` — a string identifier the client resolves to a React component (e.g. `"generic:Review"` or `"btu_building_rep_import:Process"`). The server is the single source of truth for which UI renders which step.
    - `componentProps` (optional) — a JSON-serializable object passed to the component. Generic components consume this for parameterization (`{ dataKey: 'previewData', title: '...' }`); custom components may ignore it.
- A `steps` map keyed by step id, each value a `WizardStepHandler`:

```
export const processStep: WizardStepHandler<ProcessInput, ProcessOutput> = {
  id: 'process',
  kind: 'custom',
  component: 'btu_building_rep_import:Process',  // <-- custom React component
  inputSchema: processInputSchema, // zod
  requiredPermission: 'admin',
  requiredComponent: 'sitespecific.btu',
  async handle({ wizard, input, storage, user, logger }) {
    // ...the logic that currently lives in the /process route...
    return {
      data: { processResults: results },
      nextStep: 'results',
      status: 'completed',
    };
  },
};
```

A custom server handler may declare a **generic** component instead. This is the common case for wizards that do bespoke server-side work but want a stock UI:

```
export const validateStep: WizardStepHandler<ValidateInput, ValidateOutput> = {
  id: 'validate',
  kind: 'review',
  component: 'generic:Review',
  componentProps: { dataKey: 'validationResults', title: 'Validation results' },
  inputSchema: validateInputSchema,
  requiredPermission: 'admin',
  async handle({ wizard, storage }) {
    // ...custom per-row validation logic using storage.*...
    return { data: { validationResults }, nextStep: 'process' };
  },
};
```

The plugin's `index.ts` exports `{ type, manifest, steps }` and registers itself with `server/plugins/wizards/registry.ts`. No Express imports anywhere in plugin code.

The `steps` map can mix custom handlers (imported from the plugin's own `steps/` folder) with prebuilt generic handlers (imported from `server/plugins/wizards/generic/`). For example:

```
import { uploadStep } from '../../generic/upload';
import { reviewStep } from '../../generic/review';
import { processStep } from './steps/process';
import { resultsStep } from '../../generic/results';

export default {
  type: 'btu_building_rep_import',
  manifest,
  steps: {
    upload: uploadStep({ inputSchema: csvUploadSchema, next: 'preview' }),
    preview: reviewStep({ dataKey: 'previewData', next: 'process' }),
    process: processStep,         // custom, lives in ./steps/process.ts
    results: resultsStep({ dataKey: 'processResults' }),
  },
};
```

The generic step factories take small config objects (input schema, `wizard.data` key to read or write, next-step id) and return a fully-formed `WizardStepHandler`. The dispatcher does not care that a handler came from `generic/` versus `./steps/` — it just invokes it.

The manifest function and the per-step `requiredPermission` are intentionally co-located on the server: the dispatcher trusts them to compute the *server's* view of what's allowed, and any client-side gating is a UI hint derived from the manifest response — never an authoritative check.

### Server dispatcher

The dispatcher exposes a **fixed** set of routes. Adding wizards never adds routes:

```
GET    /api/wizards/:id                          # returns wizard + computed manifest
POST   /api/wizards/:id/steps/:stepId            # invoke a step's JSON handler
POST   /api/wizards/:id/steps/:stepId/upload     # invoke a step's multipart handler
GET    /api/wizards/:id/steps/:stepId/download   # invoke a step's export/download
handler
```

(`POST /api/wizards` for creation and `DELETE /api/wizards/:id` for cancellation already exist and continue to be used unchanged.)

For `GET /api/wizards/:id`, the dispatcher:

1. Loads the wizard via `storage.wizards`. 404 if missing.
2. Looks up the plugin by `wizard.type`. If registered, calls `manifest({ wizard, storage, user })` and returns `{ wizard, manifest }`.
3. If the plugin is not registered (i.e. the wizard is still on the old system), returns `{ wizard }` only. The old client code path keeps working.

For `POST /api/wizards/:id/steps/:stepId`, the dispatcher must:

1. Load the wizard. 404 if missing.
2. Look up the plugin. 404 if not registered.
3. Look up the step by `:stepId`. 404 if not declared.
4. Enforce the step's `requiredPermission` and the wizard's `requiredComponent` (and any step-specific `requiredComponent`) using the same middleware the rest of the app uses. **One place** enforces this for all wizards.
5. Validate `req.body` against the step's `inputSchema`.
6. Call the handler with `{ wizard, input, storage, user, logger, requestId }`. The handler returns `{ data?, nextStep?, status? }` (all optional).
7. The dispatcher (not the handler) applies the patch via `storage.wizards.update(...)`, merging `data` correctly so prior-step output is not clobbered.

8. Return the handler's `data` to the client. Standard error handling + Winston logging happen here, once, for every wizard.

For `POST …/upload`, the dispatcher mounts `multer` (memory storage, configurable size limit) only for steps whose handler declares `kind: 'upload'`. The handler receives `input.file` in its context. No per-wizard module wires up multer.

For `GET …/download`, the handler returns a streamable body + headers descriptor that the dispatcher writes to the response. This is for report / export style steps.

### Client `WizardView`

The client `WizardView` (replacing the per-wizard switch in the existing `wizard-view.tsx`) is purely data-driven:

1. Fetches `GET /api/wizards/:id` via TanStack Query. The response includes both the wizard and the computed manifest.
2. Renders the progress strip from `manifest.steps` (id, name, state).
3. Looks at `wizard.currentStep`, finds the matching `step.id` in the manifest, reads the manifest entry's `component` identifier, and looks the component up in the flat auto-discovered registry. The registry is built at module load using Vite's `import.meta.glob`:

```
const modules = import.meta.glob('./*/*.tsx', { eager: true });
// './generic/Review.tsx' -> "generic:Review"
// './btu_building_rep_import/Process.tsx' -> "btu_building_rep_import:Process"
```

   There is no wizard-specific lookup logic and no per-wizard `index.ts`. The server's `component` string is the authoritative pointer. If it does not resolve, the registry throws at first render rather than silently rendering nothing — discovery failures should be loud.

4. Provides each step component with:
   - The current `wizard` object.
   - The manifest entry for this step (so the component knows its own kind, name, etc. if needed for display).
   - The manifest entry's `componentProps`, passed straight through so generic components can be parameterized from the server.
   - A typed `useStepAction(stepId)` hook that posts to `/api/wizards/:id/steps/:stepId` (or `…/upload` when the manifest entry declares `kind: 'upload'`), invalidates the wizard query on success, and returns standard `{`

`mutate, isPending, error }`.

Step components are pure React. They do **not** know their own URL. They call `useStepAction('process').mutate({ ... })` and the framework handles routing.

Generic components should publish a Zod schema for their expected `componentProps`. The registry validates the server-supplied `componentProps` against that schema at render time and surfaces a clear error when they don't match, so server/client mismatches fail fast at developer time rather than producing silent UI bugs.

If the manifest is missing from the response (i.e. the wizard type is not registered in the new framework), `WizardView` falls back to the existing per-wizard rendering path so unmigrated wizards keep working.

### Step kinds and the generic step library

`kind` and `component` are two **independent** axes:

- `kind` lives on the server and tells the **dispatcher** how to transport this step's invocation: most steps use the standard JSON POST endpoint, but `kind: 'upload'` causes the dispatcher to route through the multipart/multer endpoint instead. That's basically all `kind` does today.
- `component` lives in the manifest response and tells the **client** which React component to render. It is fully decoupled from `kind`: a step with `kind: 'custom'` can declare `component: 'generic:Review'`, and a step with `kind: 'review'` can declare a custom component if the prebuilt one isn't enough.

The kinds:

- `'form'` — reserved for a future schema-driven renderer. **Not implemented in this spike.** Including it in the type union now keeps the door open.
- `'upload'` — dispatcher mounts multer for this step's POST. The step's handler receives `input.file`. The prebuilt `generic/upload.ts` handler validates the file, stashes parsed output on `wizard.data` under a configurable key, and advances to the next step.
- `'review'` — no transport difference from a plain JSON step; the kind exists mostly as a hint to the prebuilt `generic/Review.tsx` component. The prebuilt `generic/review.ts` handler is the trivial "advance to next step" no-op; the work was already done by the previous step.
- `'results'` — terminal display. Prebuilt `generic/results.ts` is a no-op handler that marks the wizard `completed`; `generic/Results.tsx` renders summary counts and any per-row errors from a configurable `wizard.data` key.

- `'custom'` — escape hatch. The handler is hand-written. The component may be hand-written *or* one of the generic ones — that's a separate decision.

For the spike, only `upload`, `review`, `results`, and `custom` need to actually work. `form` is type-union-only.

**The point of the generic library**: most wizards in the codebase are CSV-import or report-style flows where the steps follow the same pattern (upload → map → validate → process → review/results), with only a couple of steps doing wizard-specific server-side work. Under the new framework:

- Generic *handlers* (`generic/upload.ts`, `generic/review.ts`, `generic/results.ts`) eliminate boilerplate on the server when the step's behavior is purely "validate input and stash it on `wizard.data`."
- Generic *components* (`generic/Upload.tsx`, `generic/Review.tsx`, `generic/Results.tsx`) eliminate React on the client when the step's UI is a stock upload form, review table, or results summary — **even when the server-side handler is fully custom**, which is your Map / Validate / Process / Review case.

The existing per-wizard module files are typically 200–400 lines of boilerplate around 50 lines of actual domain logic; this collapses that ratio. A typical new wizard might end up as one server handler file for the one step that does real work, with everything else composed from generics on both sides — and zero files in `client/src/plugins/wizards/<name>/`.

## Coexistence with the old system

- The new dispatcher routes live alongside the old per-wizard routes. They do not conflict — the URL prefixes are different (`/api/wizards/:id/steps/...` vs. `/api/btu-building-rep-import/...`).
- The existing `wizard-view.tsx` either learns to check the client registry *first* and fall back to its existing per-wizard branches, or is split so that registered-plugin wizards go through the new `WizardView` and unmigrated wizards go through the old one. Either is fine; the implementer should pick whichever is less invasive.
- The existing `server/wizards/types/` base class and `getSteps()` / `getStatuses()` mechanism stay in place for unmigrated wizards. The spike should not refactor `BaseWizard`.

## Auditability — the actual goal

A reviewer asking *"can a user without the right permission invoke a wizard step?"* should, after this spike (and after the eventual full migration), be able to answer that question by reading:

1. The dispatcher (one file).
2. The step handler declarations under `server/plugins/wizards/`, each of which states its own `requiredPermission` next to its handler.

…and **nothing else**. That is the success criterion the framework needs to serve. Any design decision that compromises it should be reconsidered.

## Steps

1. **Inventory the existing endpoints.** Before writing any new code, list every wizard-related HTTP route currently registered under `server/modules/` and (where applicable) under `server/routes.ts`. Group them by wizard type. Confirm each one fits cleanly into the proposed dispatcher contract (`steps/:stepId`, `steps/:stepId/upload`, `steps/:stepId/download`). If any endpoint genuinely does not fit, surface it for discussion *before* designing around it — the answer is to widen the dispatcher contract, not to keep a bespoke route.
2. **Design the manifest + handler types.** Add the `WizardManifest`, `WizardStepManifest`, `WizardStepKind`, `WizardStepHandler<Input, Output>`, and `WizardPlugin` types in `server/plugins/wizards/types.ts`. Decide and document how `wizard.data` is merged on step completion (deep merge vs. shallow, behavior on conflict).
3. **Build the server dispatcher and registry.** Implement the fixed route set above in `server/plugins/wizards/dispatcher.ts`. Implement the wizard plugin registry at `server/plugins/wizards/registry.ts`, mirroring the conventions of the four existing plugin systems under `server/plugins/` (note the intentional `plugins/plugins/` nesting). Ensure the dispatcher enforces `requiredPermission` and `requiredComponent` exactly the same way as the rest of the app. Centralize multer setup for `kind: 'upload'` steps. Centralize wizard-state persistence (`storage.wizards.update`) so handlers never call it directly.
4. **Build the generic step library on the server.** Implement the three prebuilt step handlers under `server/plugins/wizards/generic/`: `upload.ts`, `review.ts`, `results.ts`. Each is a small factory that takes a config object and returns a `WizardStepHandler`. These have to be useful enough for the BTU Building Rep Import port to consume at least one of them; that's the bar for "done."
5. **Build the client `WizardView`, auto-discovered registry, and generic components.** Implement the auto-discovered component registry at `client/src/plugins/wizards/registry.ts` using `import.meta.glob` over sibling folders; the registry maps `"<namespace>:<ComponentName>"` to a React component. Implement the

generic `WizardView` at `client/src/plugins/wizards/WizardView.tsx` that fetches the wizard + manifest and renders step components by looking up the manifest entry's `component` identifier in the registry — there is no wizard-specific lookup logic. The registry must throw a clear error at first render when an identifier doesn't resolve. Implement the `useStepAction(stepId)` hook that hides URL construction from step components. Implement default React components for the generic step ids under `client/src/plugins/wizards/generic/` (`Upload.tsx`, `Review.tsx`, `Results.tsx`), each publishing a Zod schema for its expected `componentProps`. Ensure unmigrated wizards still go through the existing rendering path.

6. **Port `btu_building_rep_import` end-to-end.** Create the server plugin folder at `server/plugins/wizards/plugins/btu_building_rep_import/`. Compose the wizard's step map using generic handlers where possible and custom handlers where required — the `process` step needs custom server-side logic (creating steward assignments); the others can likely use generic handlers. For each step, declare a `component` identifier in the handler (either `"generic:Upload"` / `"generic:Review"` / `"generic:Results"` for stock UI, or `"btu_building_rep_import:<ComponentName>"` for any step that needs custom UI). Refactor the moved handlers to call `storage.*` only and return the new `{ data, nextStep, status }` shape instead of mutating wizard state directly. Create custom React components under `client/src/plugins/wizards/btu_building_rep_import/` **only** for the steps where the generic component is genuinely insufficient (likely the preview step, because it has a wizard-specific table layout). For an all-generic step, ship zero client-side files. Delete the old per-wizard module file and the old type-class file. Confirm the wizard works end-to-end (upload CSV, preview, process, results) from a logged-in admin user.

7. **Verify the other 17 wizards still work.** Smoke-test at least two unmigrated wizards from different areas (e.g. one BTU import wizard and one GBHET report wizard) to confirm coexistence. No code changes expected; this is a check.

8. **Document the convention.** Add a short "Wizards" section to `replit.md` describing the layout (server plugin under `server/plugins/wizards/plugins/<name>/` + client step components under `client/src/plugins/wizards/<name>/` + shared generic libraries under the respective `generic/` folders), pointing at the manifest response as the source of truth for the client, listing the available step kinds, and naming `btu_building_rep_import` as the canonical example. Note explicitly that the storage-layer rule still applies to plugin handlers.

9. **Restart the `Start application` workflow** as the last step before handing back, per the repo-wide rule about server / shared changes.

## Relevant files

- `server/wizards/types/btu_building_rep_import.ts`
- `server/wizards/base.ts`
- `server/wizards/registry.ts`
- `server/wizards/index.ts`
- `server/modules/sitespecific/btu/building-rep-import.ts`
- `server/plugins/trust/eligibility`
- `server/plugins/ledger/charge`
- `server/plugins/dispatch/eligibility`
- `server/plugins/dashboard`
- `client/src/plugins`
- `client/src/pages/wizard-view.tsx`
- `client/src/pages/sitespecific/btu/building-rep-import.tsx`
- `server/routes.ts`
- `replit.md`